

# LANDMARKS-LCA ALGORITHM AND GRAPH ANALYSIS

Выполнили:

- Баранов Андрей
- Макухин Егор

Группа 25Б-72мм

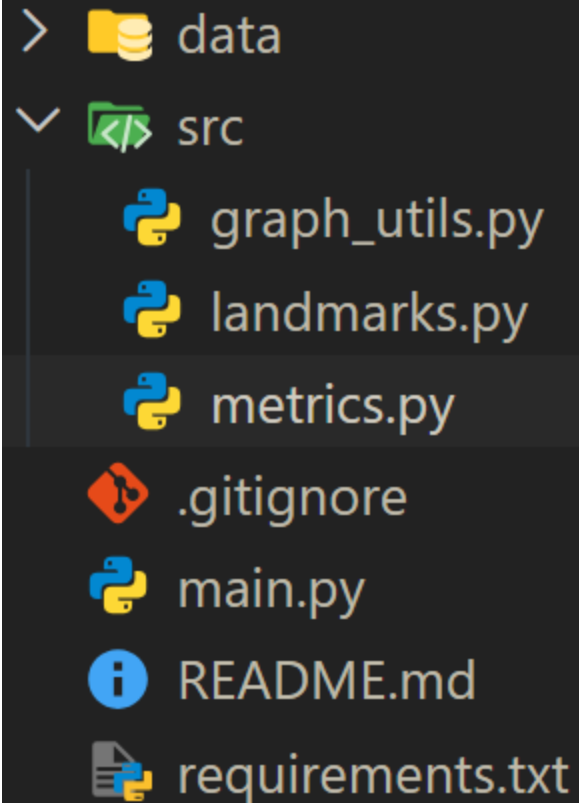
# СТРУКТУРА ПРОЕКТА

---

Датасеты:

Основной код:

Интерфейс:



```
> data
  src
    graph_utils.py
    landmarks.py
    metrics.py
  .gitignore
  main.py
  README.md
  requirements.txt
```

The image shows a file explorer window with a dark background. It displays the project structure. At the top, there is a folder icon and the name 'data'. Below it, a folder icon with a code symbol and the name 'src' is shown, with a vertical line indicating it is expanded. Inside the 'src' folder, three Python files are listed: 'graph\_utils.py', 'landmarks.py', and 'metrics.py', each with a Python logo icon. Below the 'src' folder, there are four files: '.gitignore' with a Git logo icon, 'main.py' with a Python logo icon, 'README.md' with an information icon, and 'requirements.txt' with a document icon.

# НАЧНЁМ С РЕАЛИЗАЦИИ ОСНОВНОГО АЛГОРИТМА:

SRC/LANDMARKS.PY (В ИНТЕРФЕЙСЕ "2"):

```
class LandmarkEstimator:
    def __init__(self, graph):
        self.graph = graph
        self.landmarks = []
        self.distances = {}          # {landmark: {node: dist}}
        self.spt_parents = {}        # {landmark: {node: parent}}
        self.spt_depths = {}         # {landmark: {node: depth}}
```

---

```
def _build_spt(self, landmark):
    """Построение Shortest Path Tree (SPT) через BFS"""
    dists = {landmark: 0}
    parents = {landmark: None}
    depths = {landmark: 0}
    queue = deque([landmark])

    while queue:
        u = queue.popleft()
        for v in self.graph.get(u, set()):
            if v not in dists:
                dists[v] = dists[u] + 1
                parents[v] = u
                depths[v] = depths[u] + 1
                queue.append(v)
    self.distances[landmark] = dists
    self.spt_parents[landmark] = parents
    self.spt_depths[landmark] = depths
```

## ПОСТРОЕНИЕ SPT:

# ВЫБОР LANDMARKS:

---

```
def select_landmarks(self, num_landmarks=20, method='degree'):
    """Пункт 2. Стратегии выбора ориентиров: 'random' или 'degree'"""
    if method == 'random':
        self.landmarks = random.sample(list(self.graph.keys()), min(num_landmarks, len(self.graph)))
    elif method == 'degree':
        sorted_nodes = sorted(self.graph.keys(), key=lambda n: len(self.graph[n]), reverse=True)
        self.landmarks = sorted_nodes[:num_landmarks]

    for l in self.landmarks:
        self._build_spt(l)
```

```
def estimate_basic(self, u, v):
    """Алгоритм Landmarks-Basic (Верхняя оценка по неравенству треугольника)"""
    min_dist = float('inf')
    for l in self.landmarks:
        dists = self.distances[l]
        if u in dists and v in dists:
            min_dist = min(min_dist, dists[u] + dists[v])
    return min_dist if min_dist != float('inf') else -1
```

---

## КЛАССИЧЕСКИЙ LANDMARKS-ALGORITHM:

# ПОИСК LCA (LOWER COMMON ANCESTOR):

---

```
def _find_lca(self, l, u, v):  
    """Поиск Наименьшего общего предка (LCA) в дереве SPT ориентира l"""  
    parents = self.spt_parents[l]  
    depths = self.spt_depths[l]  
    if u not in depths or v not in depths: return None  
  
    curr_u, curr_v = u, v  
    while depths[curr_u] > depths[curr_v]:  
        curr_u = parents[curr_u]  
    while depths[curr_v] > depths[curr_u]:  
        curr_v = parents[curr_v]  
    while curr_u != curr_v:  
        curr_u = parents[curr_u]  
        curr_v = parents[curr_v]  
    return curr_u
```

# LANDMARKS-LCA ALGORITHM:

---

```
def estimate_lca(self, u, v):  
    """  
    Модификация Landmarks-LCA.  
    Формула:  $d(u,v) = \text{depth}(u) + \text{depth}(v) - 2 * \text{depth}(\text{LCA}(u,v))$   
    """  
  
    min_dist = float('inf')  
    for l in self.landmarks:  
        depths = self.spt_depths[l]  
        if u in depths and v in depths:  
            lca = self._find_lca(l, u, v)  
            if lca is not None:  
                dist = depths[u] + depths[v] - 2 * depths[lca]  
                min_dist = min(min_dist, dist)  
    return min_dist if min_dist != float('inf') else -1
```



# РЕЗУЛЬТАТЫ РАБОТЫ:

## ОРИЕНТИРОВАННЫЙ (WEB-GOOGLE):

| [3/4] РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ: |    |                 |                  |           |             |                |         |           |             |
|--------------------------------|----|-----------------|------------------|-----------|-------------|----------------|---------|-----------|-------------|
| Метод                          | K  | Время пре-проц. | Запрос Basic(мс) | MAE Basic | Точн. Basic | Запрос LCA(мс) | MAE LCA | Точн. LCA | Недооц. LCA |
| random                         | 10 | 18.50           | 0.0058           | 3.858     | 0.0%        | 0.0220         | 0.332   | 75.3%     | 0           |
| random                         | 30 | 51.63           | 0.0127           | 3.234     | 0.0%        | 0.0630         | 0.218   | 81.6%     | 0           |
| random                         | 50 | 87.54           | 0.0225           | 3.037     | 0.0%        | 0.1175         | 0.137   | 87.4%     | 0           |
| degree                         | 10 | 18.27           | 0.0050           | 1.246     | 25.2%       | 0.0210         | 0.321   | 76.4%     | 0           |
| degree                         | 30 | 54.26           | 0.0145           | 0.677     | 55.8%       | 0.0642         | 0.181   | 84.7%     | 0           |
| degree                         | 50 | 94.89           | 0.0225           | 0.326     | 76.6%       | 0.1136         | 0.119   | 89.6%     | 0           |

## НЕОРИЕНТИРОВАННЫЙ (CA-GRQC):

| [3/3] РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ: |    |                 |                  |           |             |                |         |           |             |
|--------------------------------|----|-----------------|------------------|-----------|-------------|----------------|---------|-----------|-------------|
| Метод                          | K  | Время пре-проц. | Запрос Basic(мс) | MAE Basic | Точн. Basic | Запрос LCA(мс) | MAE LCA | Точн. LCA | Недооц. LCA |
| random                         | 10 | 0.02            | 0.0020           | 3.099     | 3.0%        | 0.0070         | 1.221   | 31.3%     | 0           |
| random                         | 30 | 0.06            | 0.0061           | 2.178     | 3.1%        | 0.0210         | 0.361   | 69.3%     | 0           |
| random                         | 50 | 0.12            | 0.0150           | 1.271     | 21.9%       | 0.0671         | 0.149   | 85.7%     | 0           |
| degree                         | 10 | 0.03            | 0.0030           | 1.855     | 17.7%       | 0.0116         | 1.376   | 28.3%     | 0           |
| degree                         | 30 | 0.09            | 0.0131           | 1.554     | 21.3%       | 0.0380         | 0.965   | 39.3%     | 0           |
| degree                         | 50 | 0.14            | 0.0160           | 1.114     | 35.1%       | 0.0691         | 0.635   | 54.4%     | 0           |



---

# SRC/METRICS.PY - ФАЙЛ С РЕАЛИЗАЦИЕЙ ПУНКТА 1

## SRC/GRAPH-UTILS.PY - ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ К METRICS.PY

### GRAPH-UTILS FUNCTIONS:

- LOAD\_GRAPH(FILEPATH, DELIMITER=NONE, IS\_DIRECTED=FALSE)
- GET\_WEAKLY\_CONNECTED\_COMPONENTS(GRAPH, REV\_GRAPH=NONE, ALL\_NODES=NONE)
- GET\_STRONGLY\_CONNECTED\_COMPONENTS(GRAPH, REV\_GRAPH, ALL\_NODES)
- BFS\_DISTANCES(GRAPH, START)

### METRICS FUNCTIONS:

- МНОГО-МНОГО ВСЕГО...

## В ИНТЕРФЕЙСЕ "1"

---

```
=== КЛИЕНТ ДЕМОНСТРАЦИИ ПРОЕКТА: LANDMARKS-LCA ===
```

```
Введите путь к файлу датасета (или наберите 'test' для синтетики): data/web-Google.txt
```

```
Граф ориентированный? (y/n): y
```

```
МЕНЮ УПРАВЛЕНИЯ:
```

1. Запустить полный структурный анализ (Часть 1)
2. Оценить расстояние между конкретной парой (Landmarks Basic vs LCA)
3. Выход

```
Выберите действие: █
```

---

# ИНТЕРФЕЙС - MAIN.PY:

# РЕЗУЛЬТАТЫ РАБОТЫ ИНТЕРФЕЙСА:

Вычисление базовых характеристик...

Вершин (V): 875713, Рёбер (E): 5105039, Плотность: 0.000007

Компонент слабой связности: 2746

Доля вершин в крупнейшей WCC: 97.73%

Компонент сильной связности: 371764

Доля вершин в крупнейшей SCC: 49.65%

[Оценка диаметра и перцентилей для крупнейшей компоненты...]

Диаметр (Double Sweep): 33

90-й перцентиль (Random): 15.0

90-й перцентиль (Snowball): 4.0

[Вычисление кластеризации и треугольников...]

Треугольников: 4655780

Глобальный CC (по формуле): 0.39384

Средний CC: 0.47625

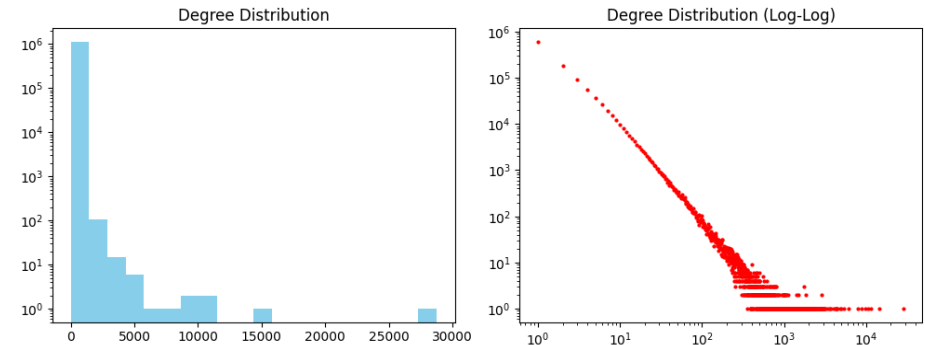
Степени: Min=0, Max=456, Avg=6.90 (График сохранен в plots/)

[Симуляция удаления x% узлов (Пункт 1.В)]

Удаление 10%: Случайное = 73.47%, Хабы = 38.91%

Удаление 30%: Случайное = 55.06%, Хабы = 0.06%

"1"



Введите вершину U: 824020

Введите вершину V: 748615

[Результаты оценки d(824020, 748615)]:

-> Точное расстояние (BFS): 4

-> Оценка Landmarks-Basic: 5

-> Оценка Модификации Landmarks-LCA: 4

"2"